Go as Scripts with Caching

Goals, failures, and working shebang patterns for `go run` and `gx run`.

- 2026-02-18
- *A frustrating dialogue between Abram and a clanker.*

Our Goals

- Run small Go programs as if they were scripts in `~/bin`.
- Use a **single `.go` file per script**, no companion wrappers.
- Use `/usr/bin/env` in the header so tools come from `$PATH`.[1][2]
- Get **caching** of compiled binaries to avoid paying full `go run` cost on each invocation.[3][4]

---

[1] Stack Overflow – "What's the appropriate Go shebang line?". https://stackoverflow.com/questions/7707178/whats-the-appropriate-go-shebang-line

[2] Baeldung – The Difference Between #!/usr/bin/bash and #!/usr/bin/env. https://www.baeldung.com/linux/bash-shebang-lines

[3] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

[4] Go project (golang/go) – Issue #71733: go run and go tool are slower than directly executing cached binary. https://github.com/golang/go/issues/71733

Rob Pike's Opinion[5]:

I have never said it cannot be done. I have always said it should not be done, and I have explained why.

"Useful" is not an argument for a feature. All features are useful; otherwise they would not be features. The issue is whether the feature justifies its cost. That is a judgement call, and my judgement is, no. Running compilers and linkers, doing megabytes of I/O, and creating temporary binary files is not a justifiable expense for running a small program. For large programs, the amortization is even more in favor of not doing this.

I am firmly against adding a feature to Go that encourages abuse of resources.

If you want the feature, use gorun or an equivalent wrapper program. That's what it's for. I think believe [sic] gorun is a mistake, but I'm not stopping you from installing it and using it as you see fit.

[5] Rob Pike – Golang-nuts discussion https://groups.google.com/g/golang-nuts/c/iGHWoUQFHjg/m/_dbLKomrPmUJ

Initial Attempt: go-run-cache & gobin

- Tried to use `go-run-cache`, but the modern repo actually builds `gobin`, not the old `go-run-cache` binary described in older docs.[6]
- `gobin` wants a module context (`go.mod`) and is aimed at running module-aware tools, not many standalone `.go` files in a `bin` directory.[7][8]
- This broke the "pile of single-file scripts with no per-script `go.mod`" constraint.

---

[6] knaka – go-run-cache command on pkg.go.dev. https://pkg.go.dev/github.com/knaka/go-run-cache

[7] Alex Edwards – Using go run to manage tool dependencies. https://www.alexedwards.net/blog/using-go-run-to-manage-tool-dependencies

[8] Go project – Go Modules Reference. https://go.dev/ref/mod

Alternative: gx Script Runner

- Found `gx` (`slyt3/gx`): "Run Go files like scripts with smart caching and auto-reload."[9]
- `gx run` compiles a script once, caches the binary, and reuses it for subsequent runs, avoiding repeated `go run` overhead.[10][11]
- This fits the caching requirement while still treating `.go` files like scripts.

[9] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

[10] Go project (golang/go) – Issue #71733: go run and go tool are slower than directly executing cached binary. https://github.com/golang/go/issues/71733

[11] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

What Didn't Work

Direct gx Shebang

- Naively tried a classic shebang:

```
#!/usr/bin/env -S gx run
package main
...
```

- Go does **not** allow # at the start of the file; the compiler reports `illegal character U+0023 '#'`.[12]
- gx does not strip a `#!` line, so it passes the # straight through to the Go parser, which fails.[13][14]

[12] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

[13] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

[14] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

What Didn't Work

Simple gx Shebang via env

- Also tried:

```
#!/usr/bin/env gx run
```

- `/usr/bin/env` treats `gx run` as a single program name (`"gx run"`), which doesn't exist.[15]
- Newer `env` versions suggest using `-S` to split arguments safely in shebang lines, but even with that, the # still breaks Go parsing if it's the very first character of the file.[16][17][18]

---

[15] J. Hermann – Shell Scripts: env-shebang with Arguments. https://jhermann.github.io/blog/linux/know-how/2020/02/28/env_with_arguments.html

[16] sul.im – How to use multiple arguments in shebang lines. https://sul.im/til/2024/10/24-env-split-string/

[17] J. Hermann – Shell Scripts: env-shebang with Arguments. https://jhermann.github.io/blog/linux/know-how/2020/02/28/env_with_arguments.html

[18] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

Working Pattern: go run "Comment Shebang"

- Use a **comment** line that is valid to both the shell and Go:

```go
//usr/bin/env go run "$0" "$@"; exit "$?"
package main

import "fmt"

func main() {
    fmt.Println("hello from go run script")
}
```

- Go sees `//...` as a normal comment and compiles the rest.[19]
- When run as `./script.go`, the shell (via the "fallback to sh on ENOEXEC" behavior) executes that first line, effectively calling `/usr/bin/env go run "./script.go" "$@"`.[20]
- This gives a single-file, `env`-based Go script, but **without any extra caching beyond what `go run` itself does**.

[19] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

[20] Stack Overflow – "What's the appropriate Go shebang line?" (accepted

Still Missing: Explicit Caching

- `go run` recompiles on changes and uses the standard Go build cache internally, but it doesn't cache a reusable script binary in a way that makes repeated CLI invocations obviously cheap.[21][22]
- The requirement was "build, cache, and run using a single script header line," so a tool like `gx` (which explicitly reuses compiled binaries) is desirable.[23][24]

---

[21] Go project (golang/go) – Issue #33468: cmd/go: speed up 'go run' by caching binaries. https://github.com/golang/go/issues/33468

[22] Coding Explorations – Mastering Go's Build System: Efficiency Through Caching and Advanced Commands. https://www.codingexplorations.com/blog/mastering-gos-build-system-efficiency-through-caching-and-advanced-commands

[23] Go project (golang/go) – Issue #71733: go run and go tool are slower than directly executing cached binary. https://github.com/golang/go/issues/71733

[24] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

Working Pattern: gx run "Comment Shebang"

- Combine the **comment shebang idea** with `gx run` instead of `go run`:

```
//usr/bin/env -S gx run "$0" "$@"; exit "$?"
package main

import "fmt"

func main() {
    fmt.Println("hello from cached gx script")
}
```

- Go: treats the first line as a comment, so the source is valid Go.[25][26]

[25] Stack Overflow – "What's the appropriate Go shebang line?" (accepted answer by Fsmv). https://stackoverflow.com/questions/7707178/whats-the-appropriate-go-shebang-line/30082862

[26] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

Working Pattern: gx run "Comment Shebang"

- Combine the **comment shebang idea** with `gx run` instead of `go run`:

```
//usr/bin/env -S gx run "$0" "$@"; exit "$?"
...
```

- Shell: when you `chmod +x script.go` and run `./script.go`, the "text executable" fallback hands it to `sh`, which executes the first line as:

```
/usr/bin/env -S gx run "./script.go" "$@"
```

- `env -S` splits `gx run` into `gx` and `run`, so this invokes `gx run script.go ...`.[27][28]

[27] J. Hermann – Shell Scripts: env-shebang with Arguments. https://jhermann.github.io/blog/linux/know-how/2020/02/28/env_with_arguments.html

[28] sul.im – How to use multiple arguments in shebang lines. https://sul.im/til/2024/10/24-env-split-string/

How gx Provides Caching

- `gx` is designed as a **fast Go script runner with smart caching**: it hashes the script, compiles it once, and reuses the resulting binary until the inputs change.[29][30]
- With the comment-shebang line, every invocation of `./script.go` is effectively `gx run script.go`, so:
  - First run: compile + cache + execute.
  - Later runs: hit cached binary, skipping the heavy toolchain steps.[31][32]

[29] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

[30] Go project (golang/go) – Issue #71733: go run and go tool are slower than directly executing cached binary. https://github.com/golang/go/issues/71733

[31] Go project (golang/go) – Issue #71733: go run and go tool are slower than directly executing cached binary. https://github.com/golang/go/issues/71733

[32] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

Final Result: Single-File, Cached Go Scripts

- **Single file**: each script is just `foo.go` in your `bin` directory.
- **Header line**: the first line is a dual-purpose
  - `//usr/bin/env -S gx run "$0" "$@"; exit "$?"` comment-shebang.[33][34][35]
- **Uses env**: `/usr/bin/env` finds `gx` via `$PATH`, keeping scripts portable.[36]
- **Caching**: `gx run` gives explicit binary caching and fast re-execution.[37][38]

[33] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

[34] J. Hermann – Shell Scripts: env-shebang with Arguments. https://jhermann.github.io/blog/linux/know-how/2020/02/28/env_with_arguments.html

[35] Stack Overflow – "What's the appropriate Go shebang line?" (accepted answer by Fsmv). https://stackoverflow.com/questions/7707178/whats-the-appropriate-go-shebang-line/30082862

[36] Baeldung – The Difference Between #!/usr/bin/bash and #!/usr/bin/env. https://www.baeldung.com/linux/bash-shebang-lines

[37] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching. https://github.com/slyt3/gx

[38] Go project (golang/go) – Issue #71733: go run and go tool are slower than

Summary Slide

- Direct `#!` with Go fails: Go doesn't accept # comments.[39]
- `go run` comment-shebang works but doesn't give an obvious cached-binary workflow.[40][41]
- `gx run` plus a comment-shebang and `/usr/bin/env -S` yields:
  - Valid Go source.
  - A single, executable `.go` script.
  - Cached, fast startup via `gx`.[42][43][44]

[39] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

[40] Stack Overflow – "What's the appropriate Go shebang line?" (accepted answer by Fsmv). https://stackoverflow.com/questions/7707178/whats-the-appropriate-go-shebang-line/30082862

[41] Go Cookbook – Running a file via a shebang line. https://golangcookbook.com/chapters/running/shebang/

[42] J. Hermann – Shell Scripts: env-shebang with Arguments. https://jhermann.github.io/blog/linux/know-how/2020/02/28/env_with_arguments.html

[43] Go project (golang/go) – Issue #71733: go run and go tool are slower than directly executing cached binary. https://github.com/golang/go/issues/71733

[44] slyt3 (GitHub) – gx: Run Go files like scripts with smart caching.

Even More Referneces

4546474849505152535 4

[45] Daily Dose of Dev – Create PowerPoint Slides from Markdown with Pandoc. https://www.youtube.com/watch?v=M2BIPVWliKk

[46] golang-nuts (Google Groups) – Thread: Using go run with #!. https://groups.google.com/g/golang-nuts/c/iGHWoUQFHjg/m/_dbLKomrPmUJ

[47] GitHub / YouTube (Pandoc-related) – Markdown: Creating Presentations Using Markdown and Pandoc. https://www.youtube.com/watch?v=e-HqKSBZOXo

[48] YouTube – Making Presentations Has Never Been Easier! Markdown + Pandoc. https://www.youtube.com/watch?v=yR3Znpf_TY8

[49] Reddit /r/golang – A golang shebang line that works with go fmt. https://www.reddit.com/r/golang/comments/xucicu/a_golang_shebang_line_that_works_with_go_fmt/

[50] Pandoc project (jgm/pandoc) – Discussion #10190: How to use PowerPoint slide layouts in Markdown. https://github.com/jgm/pandoc/discussions/10190

[51] duanemay – test-go-shebang: Shebang and bash functions on Go Scripts. https://github.com/duanemay/test-go-shebang

[52] Reddit /r/golang – gosh (Go code from the shell) now supports shebang (#!) scripts. https://www.reddit.com/r/golang/comments/lsutuh/gosh_go_code_from_the_shell_now_supports_shebang/

[53] Pandoc Project – Demos. https://pandoc.org/demos.html

[54] Eyal Posener – A Story About Writing Scripts with Go. https://posener.github.io/go-shebang-story/